

Piecewise-Affine Local Volatility

Calibrating a Dupire surface from sparse quotes: model, PDE, gradients, and speed

Vol-Fitter Technical Notes, No. 04

Vol-Fitter

July 3, 2026

Abstract

This note documents the Vol-Fitter’s full local-volatility model: a continuous piecewise-affine (P_1 finite-element) local-variance surface $\nu_\theta(t, x)$, calibrated directly to sparse vanilla and variance-swap quotes through the forward Dupire PDE as an implicit pricing map. Unlike the slice models (Notes 01–03) this is a genuine two-dimensional local-volatility surface: strict positivity of the nodal variances makes the implied prices butterfly- and calendar-clean up to scheme noise, which the model *measures* rather than assumes. We derive the normalized forward Dupire equation, the P_1 parametrization and its positivity guarantee, the implicit-Euler discretization and observation operator, the variance-swap log-contract replication and its faster source-PDE alternative, and the calibration objective with its roughness, convex-wing and front-tie regularizers. We then give the sensitivity PDE and the adjoint gradient, and — the engineering heart of the note — the numerical optimizations that cut the cold fit $\sim 3\text{--}6\times$ (from $\sim 86\text{ s}$ to $\sim 15\text{--}30\text{ s}$, scaling with grid size), with recalibrations near-instant: a compiled vectorized-Thomas Dupire march ($\sim 6.5\times$), a stall-based early-stop, a parametric Dupire warm seed, and the matrix-free Gauss–Newton solver that avoids the dense SVD. A synthetic round trip recovers a known surface to 0.6 vol bps, and the production fit on a static Bloomberg fixture reaches 2.8 vol bps on SPY and 11.7 on NVDA. The note closes with the short-dated-fit fixes and the convex-wing regression that the benchmark caught.

Contents

1	The idea	3
2	The forward Dupire equation	3
3	The piecewise-affine surface	3
3.1	Parametrization	3
3.2	Grid design	4
4	The pricing map	4
4.1	Discretization	4
4.2	No-arbitrage diagnostics	5
5	Variance swaps	5
6	The calibration objective	5

7	Sensitivities and the adjoint	6
7.1	The sensitivity PDE	6
7.2	The adjoint gradient	6
8	Solvers and the speed story	6
8.1	Where the time goes	6
8.2	What worked	7
9	Short-dated calibration	7
9.1	The convex-wing regression	8
10	Worked examples	8
10.1	Synthetic round trip	8
10.2	The Bloomberg benchmark	8
11	What is original, and limitations	9
A	Hyperparameter atlas	9
B	Performance notes	10
C	Reference implementation: Dupire extraction	11

1 The idea

A slice model fits one expiry at a time and stitches expiries together with a calendar constraint. A *local-volatility* model does the opposite: it posits a single function $\sigma_{\text{loc}}(t, S)$ such that, under $dS_t = \sigma_{\text{loc}}(t, S_t) S_t dW_t$ (in forward-normalized units), the model reproduces *all* expiries and strikes at once. Dupire showed such a function exists and is unique given an arbitrage-free price surface. The Vol-Fitter follows the Andreasen–Huge philosophy: rather than *differentiate* a fitted price surface to extract σ_{loc} (numerically explosive in sparse data), it *parametrizes* σ_{loc} directly and prices *forward* through the Dupire PDE, calibrating the parameters to quotes.

Heuristic

Extracting local vol by the Dupire formula $\sigma_{\text{loc}}^2 = \partial_T C / (\frac{1}{2} K^2 \partial_{KK} C)$ divides two noisy numerical derivatives of a sparse surface — a recipe for garbage in the wings. Calibrating local vol *forward* instead never differentiates the data: it asks “what positive surface, run through the PDE, best reprices the quotes?” Positivity of the surface is imposed on the *parameters*, so every iterate is arbitrage-clean by construction — the same philosophy as LQD, one dimension up.

2 The forward Dupire equation

Work in forward-normalized strike $x = K/F_T$ and let $c(T, x)$ be the normalized call. Writing local variance $\nu(t, x) = \sigma_{\text{loc}}^2$, the forward Dupire equation is

$$\partial_T c(T, x) = \frac{1}{2} \nu(T, x) x^2 \partial_{xx} c(T, x), \quad c(0, x) = (1 - x)^+. \quad (1)$$

It is a forward parabolic PDE in (T, x) with the call payoff as initial data. Two facts make it the right object. First, $\partial_{xx} c \geq 0$ is exactly the butterfly (density) condition, and $\nu \geq 0$ keeps the equation parabolic, so a positive ν *propagates* non-negativity of the density forward in maturity. Second, monotonicity of c in T (the calendar condition) follows from (1) whenever $\nu \geq 0$, since the right-hand side has the sign of $\partial_{xx} c \geq 0$. Thus a strictly positive local-variance surface is *structurally* free of both static arbitrages, up to the discretization noise the model measures (section 4.2).

3 The piecewise-affine surface

3.1 Parametrization

The local variance is a continuous piecewise-affine (P_1) function on a tensor grid of $n_t \times n_x$ vertices (τ_i, ξ_j) ,

$$\nu_\theta(t, x) = \sum_{i,j} \theta_{ij} \phi_{ij}(t, x), \quad (2)$$

where ϕ_{ij} are the bilinear (or barycentric/Delaunay) hat functions and $\theta_{ij} = \nu(\tau_i, \xi_j) \geq 0$ are the *nodal local variances* — the calibration parameters. The flat vector is $\theta = \text{vec}(\theta)$, time-major.

Proposition 1 (Nodal positivity $\Rightarrow$ surface positivity). *A P_1 function is a convex combination of its vertex values on each cell. Hence if $\theta_{ij} \geq 0$ for all i, j , then $\nu_\theta(t, x) \geq 0$ everywhere, and $\nu_\theta > 0$ if the nodal values are bounded below by a positive constant.*

This is why the calibration enforces only *box bounds* on θ (default local-variance bounds $[0.005, 0.20]$, i.e. local vol in $[7\%, 45\%]$): positivity of the whole surface reduces to positivity of finitely many numbers. Beyond the lowest strike vertex the surface extrapolates the local variance *linearly* with a slope set by the first interior cell (`left_extrap_a`; Note 09), rising toward $x \rightarrow 0$; the right wing is flat-clamped.

3.2 Grid design

The vertex grid is delta-spaced in strike and floored per expiry so that even a narrow short-dated smile lands enough vertices on its curvature (`gridXNodes`, `gridXMinPerExpiry`; section 9). The time vertices (`gridTNodes`) carry the term structure. The grid is deliberately modest — a dozen strike vertices, ten in time — because each vertex is a free parameter and the PDE solve cost scales with the grid; the regularizer (section 6) supplies the smoothness that a coarse grid would otherwise lack.

4 The pricing map

4.1 Discretization

The forward map (1) is solved on a uniform spatial grid with step $\Delta x = 0.01$ on $x \in [0, \sim 2.5]$ (~ 250 nodes) by fully implicit (backward) Euler in maturity with step $\Delta T = 0.01$ (the local variance entering each step at the new time level t_{n+1}), optionally Rannacher-started with a couple of implicit start-up steps before Crank–Nicolson. (A separate log-strike validator, appendix A, runs the finer fixed grid.) Each step solves a tridiagonal M -matrix system

$$(I - \Delta T A_n) U^{n+1} = U^n, \quad (3)$$

where A_n encodes $\frac{1}{2}\nu x^2 \partial_{xx}$ at step n . The matrix is an M -matrix (diagonally dominant, non-positive off-diagonals), so the scheme is unconditionally stable and monotone — it cannot create spurious negative densities. The observation operator reads the normalized call at each quote’s (T, x) off the solution by interpolation.

Diagonal dominance makes pivoting unnecessary, so each step (3) is a no-pivot Thomas march — the kernel the compiled march (appendix B) accelerates $\sim 6.5\times$:

Listing 1: The no-pivot tridiagonal (Thomas) solve for one implicit step (3) (distilled from `models/localvol/affine_march.py`; matches a dense solve to 10^{-10}).

```

1  def thomas(a, b, c, d):                # a: sub-diagonal, b: diagonal, c:
    super-diagonal
2      n = len(d); cp = np.empty(n); dp = np.empty(n)
3      cp[0], dp[0] = c[0]/b[0], d[0]/b[0]
4      for i in range(1, n):              # forward sweep: eliminate the sub-
        diagonal
5          m = b[i] - a[i]*cp[i-1]
6          cp[i], dp[i] = c[i]/m, (d[i] - a[i]*dp[i-1])/m
7      x = np.empty(n); x[-1] = dp[-1]
8      for i in range(n-2, -1, -1):        # back substitution
9          x[i] = dp[i] - cp[i]*x[i+1]
10     return x

```

4.2 No-arbitrage diagnostics

Because the surface is only *up to scheme noise* arbitrage-free, the model measures the residual: a butterfly tolerance on $\partial_{xx}c$ ($\gtrsim -10^{-6}$ density) and a calendar tolerance on $\partial_T c$ ($< -10^{-9}$ price decrement). The model is gated behind these diagnostics rather than assuming cleanliness — a deliberate design choice.

5 Variance swaps

A variance-swap quote is the fair strike of realized variance, which for a diffusion equals the expected total variance to T . Two pricers are available. The *static* method uses the log-contract replication

$$I(T) = 2 \int_0^1 \frac{P(T, x)}{x^2} dx + 2 \int_1^\infty \frac{C(T, x)}{x^2} dx, \quad (4)$$

a trapezoid over the priced surface (half-width and node count `VS_HALF_WIDTH`, `VS_POINTS`). The faster *source-PDE* method (default-off, gated by `varSwapMethod`) solves one backward source PDE $\partial_t g + \frac{1}{2}\nu x^2 \partial_{xx} g + \nu = 0$, $g(T, \cdot) = 0$, and reads $I(T) = g(0, 1)$ — with analytic sensitivities $\partial I / \partial \theta$ from the same multi-RHS machinery, and without the $x_{\max}$ -dependence of the replication tail. Note 08 develops both in full.

6 The calibration objective

The fit minimizes a stacked least-squares residual over θ ,

$$\min_{\theta \in [\nu_{lo}, \nu_{hi}]} \frac{1}{2} \left(\underbrace{\|r_{\text{opt}}\|^2}_{\text{quotes}} + \underbrace{\|r_{\text{vs}}\|^2}_{\text{var-swaps}} + \underbrace{\lambda \|L(\theta - \theta_{\text{ref}})\|^2}_{\text{roughness}} + \underbrace{w_{\text{cvx}} \|r_{\text{cvx}}\|^2}_{\text{convex wing}} + \underbrace{w_{\text{ft}} \|r_{\text{ft}}\|^2}_{\text{front tie}} \right), \quad (5)$$

with the blocks:

- **Option residuals** $r_{\text{opt},q} = (P_q(\theta) - y_q) / \eta_q$ in vega-normalized forward-call price units (mid), or the bid–ask/haireut band hinge with a small mid anchor when band edges are present (Note 07).
- **Var-swap residuals** $r_{\text{vs}} = (I(\theta) - z) / \zeta$ (Note 08).
- **Roughness** $\sqrt{\lambda} L(\theta - \theta_{\text{ref}})$ with L the spacing-aware second-difference operator and θ_{ref} a reference surface; the time-vs-strike balance is `gridRegRho`. This is the smoothness that lets a coarse grid generalize.
- **Convex wing** hinge: a one-sided penalty enforcing local convexity of the wing, confined to vertices *below the deepest quote* (the extrapolation tail only — see section 9.1).
- **Front tie**: a light penalty tying the unconstrained $t = 0$ time row to the first (data-identified) expiry row across strikes, anchoring the front end where the shortest expiry has little coverage.

7 Sensitivities and the adjoint

7.1 The sensitivity PDE

Each parameter θ_ℓ enters (1) linearly through ν_θ , so the price sensitivity $s_\ell = \partial c / \partial \theta_\ell$ obeys the same parabolic PDE with a *source*:

$$\partial_T s_\ell = \frac{1}{2} \nu_\theta x^2 \partial_{xx} s_\ell + \frac{1}{2} \phi_\ell x^2 \partial_{xx} c, \quad s_\ell(0, \cdot) = 0. \quad (6)$$

Discretized, every s_ℓ shares the *same* tridiagonal factor as the value solve (3), differing only in the source — so the Jacobian columns are a single multi-RHS back-substitution against one factorization. This is the forward-sensitivity route, $O(N_t N_x m)$ for m active parameters.

7.2 The adjoint gradient

When only the *gradient* of the scalar objective is needed (not the full Jacobian), the discrete adjoint computes it in one forward value solve plus one backward adjoint sweep, $O(N_t N_x)$ *independent of m* . The Vol-Fitter uses the forward sensitivities for the Gauss–Newton Jacobian (it needs the columns) and keeps the adjoint as the route for very large vertex counts.

8 Solvers and the speed story

The cold fit of (5) was, at ~ 533 vertices, an ~ 86 second affair hitting the 200-evaluation cap — the single heaviest computation in the app. Driving it down is most of the engineering in this model, and the lessons are worth stating because several plausible ideas *failed*.

8.1 Where the time goes

On a cold fit the cost splits roughly as: optimizer linear algebra / dense SVD $\sim 52\%$, PDE sensitivity march $\sim 32\%$, Jacobian assembly $\sim 14\%$, value solve $\sim 2\%$. No single per-evaluation lever moves the total, because the cost is distributed — which is why four separate “faster per evaluation” attempts (coarse grid, first Numba try, Rannacher, and the first matrix-free GN) each won only ~ 1.1 – $1.2\times$ and were shelved.

8.2 What worked

Performance

The cold fit is now ~ 3 – $6\times$ faster than the original banded baseline (scaling with grid size); recalibrations are near-instant via warm start. The levers, in order of impact:

1. **Stall-based early-stop.** Track the option-block misfit and stop at the best iterate once it stalls. Because fewer evaluations multiply march, assembly *and* optimizer together, this scales the *whole* fit: measured $1.45\times$ (SPY) to $3.3\times$ (NVDA), at $+0.1$ – 0.25 vol bp.
2. **Compiled vectorized-Thomas march.** A `@njit` no-pivot Thomas solver with the sensitivity columns as the contiguous SIMD inner loop and a fused source: $\sim 6.5\times$ the LAPACK banded march, numerically exact to $\sim 10^{-15}$. The first Numba attempt (column-outer scalar Thomas) won only $1.2\times$ — the real lever was the loop order.

3. **Matrix-free Gauss–Newton.** The default solver avoids `trf`’s dense SVD (52% of an evaluation) by solving the GN step with preconditioned LSMR on matrix-free $Jv/J^\top w$ products. Once the march is cheap, the no-SVD evaluations win: $\sim 1.3\text{--}1.65\times$ over TRF. Gated to the smooth mid-fit target.
4. **Parametric Dupire warm seed.** Seed θ from the parametric surface’s local variance ($\sim 1.3\text{--}1.8\times$ on cold fits) and from the previous surface on recalibration ($\sim 38\times$).

The order-of-magnitude wins are spent; remaining levers are incremental.

Caution

Removing the dense SVD made fits *slower* at ≤ 440 vertices, because there the bottleneck is the PDE march, not the SVD — the SVD wall is a $\gtrsim 1000$ -vertex (future non-tensor “bowtie” grid) phenomenon. The clean synthetic benchmark (zero residual, in bounds) hid this: GN converged in 8 evaluations there but did *not* converge within the cap on stiff real data. The honest lesson: measure on real chains, and distrust a synthetic-only speed-up.

9 Short-dated calibration

A genuine one-week smile fit *catastrophically* (108 vol bp RMS vs the parametric ~ 47) while normal expiries fit well. A measure-first diagnosis (per-expiry side-channel counting in-range vertices, vega flooring and PDE steps) ruled out the usual suspects — vega floor, time steps, local-vol cap, prior leakage — and isolated *strike under-resolution*: the delta strike axis is sized to the *longest* expiry and clipped to the global range, so a narrow short smile landed only ~ 3 vertices on its sharpest curvature. Two fixes:

1. **Per-expiry coverage floor** (`gridXMinPerExpiry`, default 8): after the delta axis is built, split the widest in-range gaps until each expiry has at least $m_{\min}$ vertices inside *its own* traded range — adding nodes only to under-covered short fronts.
2. **Short-expiry-aware PDE step**: refine the shared PDE x -step to $0.3\times$ the smallest ATM $\sigma\sqrt{\tau}$, snapped so $x = 1$ stays a node, clamped to $[1/400, 0.01]$. Normal surfaces stay on 0.01 (byte-identical).

The one-week fit fell from $108 \rightarrow 23.5$ vol bp (now *better* than the parametric), with the long expiries byte-identical.

9.1 The convex-wing regression

The benchmark also caught a subtler bug. The convex-wing constraint originally selected *every* vertex in the deep wing regardless of data; at a user’s saved `gridXNodes = 20` it stacked convexity penalties onto densely-quoted put strikes and forced the *wrong* wing on low-vol SPY (NVDA’s naturally convex wing had hidden it). The fix confines `convex_cols` to vertices *below the deepest quote* — the extrapolation tail only. SPY went $25.7 \rightarrow 2.6$ vol bp. The lesson echoes Note 09: a shape constraint must not be imposed where the data already speaks.

10 Worked examples

10.1 Synthetic round trip

A known skewed local-variance surface is priced through the forward PDE, quotes are generated at four expiries, and the surface is recovered from a *flat* seed by the production calibrator. Figure 1 shows the recovered surface and the implied-vol fit: the calibrator recovers the truth to a maximum of 0.6 vol bps in 33 evaluations (1.01 s wall, 28 vertices), with the skew (higher local vol at low strike) faithfully reconstructed.

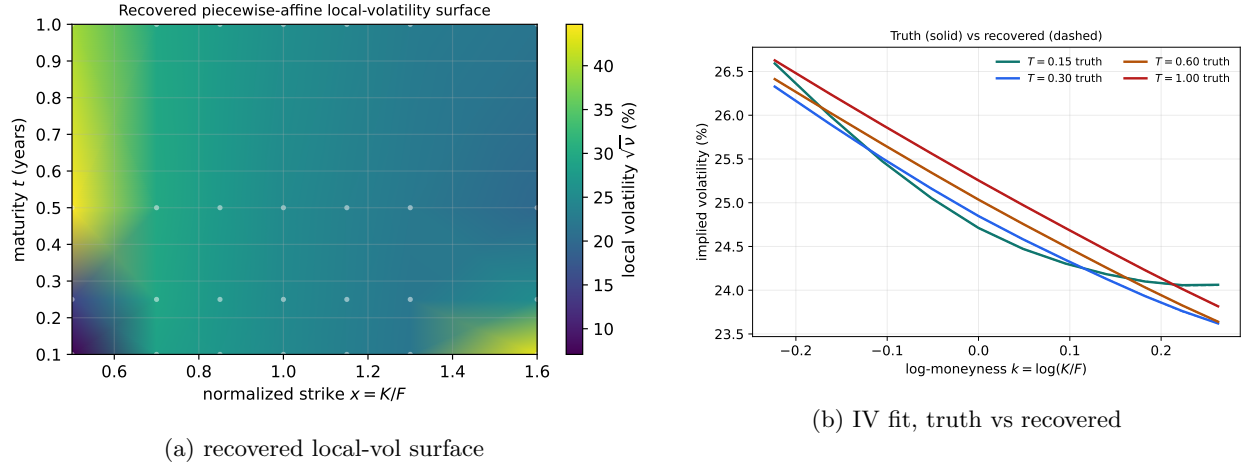


Figure 1: Synthetic round trip through the real calibrator: the skewed surface is recovered to 0.6 vol bps from a flat seed.

10.2 The Bloomberg benchmark

On the static Bloomberg fixture the production fit reaches the quality in table 1 and fig. 2: 2.8 vol bps surface RMS on SPY and 11.7 on NVDA, with no node at a bound on the easy expiries. The harder NVDA short-dated expiries carry the larger error — the regime section 9 addresses.

Table 1: Production local-vol fit on the Bloomberg fixture (fresh run).

Name	surface RMS (bp)	worst expiry (bp)
SPY	2.8	4.4
NVDA	11.7	12.4

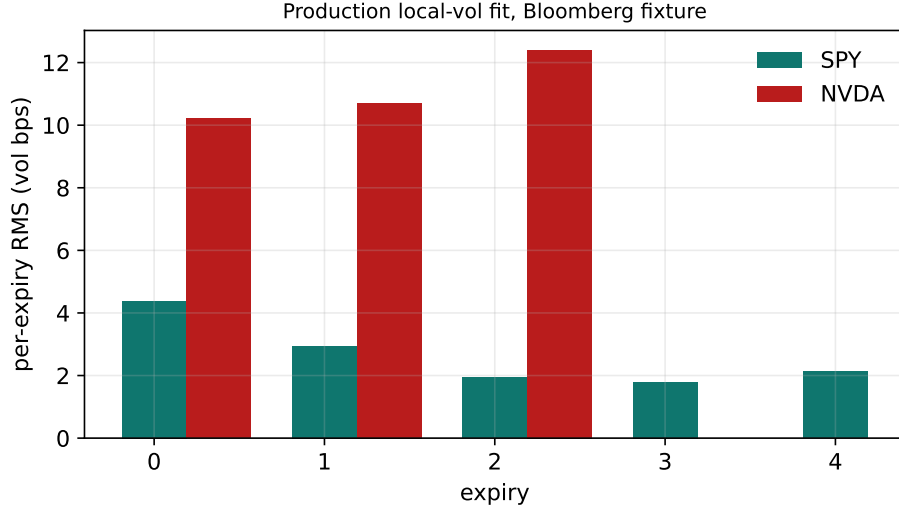


Figure 2: Per-expiry weighted RMS for SPY and NVDA on the fixture.

11 What is original, and limitations

The P_1 -positivity guarantee and the forward-calibration philosophy are Andreasen–Huge; the contributions here are engineering depth. The *vectorized-Thomas Numba march* (loop order as the lever, $6.5\times$), the *stall early-stop* that scales the whole fit, the *matrix-free GN* correctly scoped to where the SVD actually dominates, the *source-PDE var-swap* with analytic sensitivities, and the *measure-first* short-dated diagnosis are each non-obvious and were validated against a real benchmark that twice caught regressions a synthetic would have missed. Limitations: the model is a pure diffusion (no jumps), so a true short-dated event spike is only approximated; the tensor grid wastes vertices in the corners (the future non-tensor bowtie grid is where GN’s no-SVD advantage would finally dominate); and the fit, though much faster, remains the heaviest single computation in the app.

A Hyperparameter atlas

Table 2: Local-volatility hyperparameters.

Knob	Default	Role
<i>Surfaced (OptionsSettings)</i>		
gridXNodes	12	Strike vertices of the variance surface.
gridXMinPerExpiry	8	Min in-range strike vertices per expiry (section 9).
gridTNodes	10	Maturity vertices.
gridRegLambda λ	10^{-2}	Roughness penalty strength.
gridRegRho ρ	1.0	Time-vs-strike roughness balance.
convexWingWeight	10^3	Convex-wing hinge weight (tail only).
frontTieWeight	10^{-2}	Front-end tie-to-ATM weight.

Knob	Default	Role
<code>lvVolCapMult</code>	3.0	Local-vol cap as a multiple of max implied vol.
<code>leftWingSlopeMult</code>	1.5	Left local-variance extrapolation slope multiple.
<code>varSwapMethod</code>	<code>static</code>	Var-swap pricer: replication or source-PDE.
<code>lvSolver</code>	<code>gn</code>	Gauss–Newton (default) or <code>trf</code> .
<code>lvFastKernel</code>	<code>true</code>	Numba vectorized-Thomas march.
<code>lvEarlyStop</code>	<code>true</code>	Stall-based early stop.
<code>timeScheme</code>	<code>implicit</code>	Implicit Euler or Rannacher/CN.
<i>Hidden (affine-calibration PDE / solver)</i>		
affine PDE step	$\Delta x = 0.01$	Spatial step on $x \in [0, \sim 2.5]$ (~ 250 nodes).
affine time step	$\Delta T = 0.01$	Implicit-Euler maturity step.
<code>rannacher_steps</code>	2	Implicit start-up steps before CN (opt-in scheme).
short-dated step	$0.3 \sigma \sqrt{\tau}$	Refined Δx floor, clamped $[1/400, 0.01]$ (section 9).
<code>VAR_FLOOR</code>	10^{-6}	Dupire local-variance floor.
variance bounds	$[0.005, 0.20]$	Nodal local-variance box (vol $[7\%, 45\%]$).
<code>x_scale</code>	<code>jac</code>	Optimizer column scaling.
<code>xtol/ftol/gtol</code>	10^{-8}	Optimizer tolerances.
<code>max_nfev</code>	200	Evaluation cap.
<code>stall_window/rtol</code>	12–18 / $\sim 3\text{--}5 \times 10^{-3}$	Early-stop window/tolerance.
<i>Hidden (separate log-strike validator, <code>pde.py</code>)</i>		
<code>DEFAULT_N_K</code>	1201	Validator spatial nodes.
<code>DEFAULT_DT_MAX</code>	1/400	Validator max time step.
<code>N_RANNACHER</code>	4	Validator Rannacher quarter-steps.
<code>SPAN_SD / SPAN_MIN</code>	7.5 / 0.6	Validator grid span (sd / floor).

B Performance notes

1. **Numba march** (`lvFastKernel`). No-pivot factor-once Thomas with the m sensitivity columns as the contiguous inner SIMD loop and fused source: $6.1\text{--}6.9\times$ vs LAPACK `dgbstv`, exact to $\sim 10^{-15}$. Graceful banded fallback if Numba is absent.
2. **Early stop** (`lvEarlyStop`). Best-iterate stop on option-block stall; scales the whole fit, $1.45\times$ (SPY) to $3.3\times$ (NVDA), $+0.1\text{--}0.25$ vol bp. `stall_window=0` is byte-identical.
3. **Matrix-free GN** (`lvSolver=gn`). Preconditioned LSMR on $Jv/J^\top w$; avoids the dense SVD. $1.3\text{--}1.65\times$ over TRF at tensor-grid sizes; gated to the smooth mid target.
4. **Warm starts**. Previous-surface seed ($\sim 38\times$ recal) and parametric Dupire cold seed ($1.3\text{--}1.8\times$).
5. **Shelved (documented)**: coarse grid (biases θ), first Numba attempt (wrong loop order), Rannacher ($1.1\times$ + arb risk), GN for the non-smooth band objective, thread/process parallelism (GIL). Four distributed-cost dead-ends confirming the cost is spread, not localized.

C Reference implementation: Dupire extraction

Forward calibration never differentiates the data (section 2), but the inverse map — read a local-variance grid off an *already-fitted* total-variance surface $w(k, T)$ — is Gatheral’s formula, used for grid export and the graph LV projection (Note 14). The denominator g is the butterfly function: $g \leq 0$ marks strike arbitrage in the implied surface, where no positive local variance can reproduce it, returned `nan`. Matches production to 10^{-13} .

Listing 2: Dupire/Gatheral local variance from an implied total-variance surface (distilled from `models/localvol/dupire.py`).

```
1 def dupire_local_variance(k, w, wk, wkk, wt):    # w=w(k,T) with its k,  
    kk, T derivatives  
2     r = k/w  
3     g = 1 - r*wk + 0.25*(-0.25 - 1/w + r*r)*wk*wk + 0.5*wkk    #  
    butterfly denominator  
4     return np.where((w > 0) & (g > 0), wt/np.where(g > 0, g, 1), np.  
    nan)    # nan if g <= 0
```

References

- [1] B. Dupire. Pricing with a smile. *Risk*, 7(1):18–20, 1994.
- [2] J. Andreasen and B. Huge. Volatility interpolation. *Risk*, March 2011.
- [3] J. Gatheral. *The Volatility Surface*. Wiley, 2006.
- [4] M. Giles and R. Carter. Convergence analysis of Crank–Nicolson and Rannacher time-marching. *J. Comput. Finance*, 9(4), 2006.